

Algoritmos_AG3

July 14, 2026

1 Algoritmos - Actividad Guiada 3

- **Nombre:** Marcos Antonio Lévano Huamaccto
- **Link a Google Colab:** [Acceder al Cuaderno Interactivo](#)
- **Repositorio GitHub:** [Ver Código Fuente](#)

2 Ejecuciones

Carga de librerías

```
[1]: #!/pip install tsplib95
import tsplib95
import random
from math import e
import math
import copy
import urllib.request
```

```
[ ]: #!/pip install requests      #Hacer llamadas http a paginas de la red
#!/pip install tsplib95      #Modulo para las instancias del problema del TSP
```

2.1 Carga de los datos del problema

```
[2]: # Descargamos el fichero de datos (Matriz de distancias)
file = "swiss42.tsp" ;
urllib.request.urlretrieve("https://raw.githubusercontent.com/mastqe/tsplib/
↪refs/heads/master/swiss42.tsp", file )
```

```
[2]: ('swiss42.tsp', <http.client.HTTPMessage at 0x2048944da90>)
```

```
[3]: problema = tsplib95.load(file)
nodos = list(problema.get_nodes())
aristas = list(problema.get_edges())

print(problema) # Descripción estándar
print(nodos) # Nodos 0, 1, 2 ... 42
print(aristas) # Ver https://github.com/mastqe/tsplib/blob/master/swiss42.tsp
```

NAME: swiss42
COMMENT: 42 Staedte Schweiz (Fricker)
TYPE: TSP
DIMENSION: 42
EDGE_WEIGHT_TYPE: EXPLICIT
EDGE_WEIGHT_FORMAT: FULL_MATRIX
EDGE_WEIGHT_SECTION:
0 15 30 23 32 55 33 37 92 114 92 110 96 90 74 76 82 67 72 78 82 159 122 131 206
112 57 28 43 70 65 66 37 103 84 125 129 72 126 141 183 124
15 0 34 23 27 40 19 32 93 117 88 100 87 75 63 67 71 69 62 63 96 164 132 131 212
106 44 33 51 77 75 72 52 118 99 132 132 67 139 148 186 122
30 34 0 11 18 57 36 65 62 84 64 89 76 93 95 100 104 98 57 88 99 130 100 101 179
86 51 4 18 43 45 95 45 115 93 152 159 100 112 114 153 94
23 23 11 0 11 48 26 54 70 94 69 89 75 84 84 89 92 89 54 78 99 141 111 109 190 89
44 11 29 54 56 89 47 118 96 147 151 90 122 126 163 101
32 27 18 11 0 40 20 58 67 92 61 78 65 76 83 89 91 95 43 72 110 141 116 105 190
81 34 19 35 57 63 97 58 129 107 156 158 92 129 127 161 95
55 40 57 48 40 0 23 55 96 123 78 75 62 36 56 66 63 95 37 34 137 174 156 129 224
90 15 59 75 96 103 105 91 158 139 164 156 78 169 163 191 115
33 19 36 26 20 23 0 45 85 111 75 82 69 60 63 70 71 85 44 52 115 161 136 122 210
91 25 37 54 78 81 90 68 136 116 150 147 76 148 147 180 111
37 32 65 54 58 55 45 0 124 149 118 126 113 80 42 42 49 40 87 60 94 195 158 163
242 135 65 63 79 106 101 50 66 118 104 109 103 36 160 178 218 153
92 93 62 70 67 96 85 124 0 28 29 68 63 122 148 155 156 159 67 129 148 78 80 39
129 46 82 65 55 40 61 157 97 159 135 212 221 159 110 72 95 35
114 117 84 94 92 123 111 149 28 0 54 91 88 150 174 181 182 181 95 157 159 50 65
27 102 65 110 87 73 50 68 176 112 166 142 229 241 184 99 46 69 38
92 88 64 69 61 78 75 118 29 54 0 39 34 99 134 142 141 157 44 110 161 103 109 52
154 22 63 68 66 61 81 158 107 175 151 216 219 150 137 100 115 37
110 100 89 89 78 75 82 126 68 91 39 0 14 80 129 139 135 167 39 98 187 136 148 81
186 28 61 92 97 98 117 173 134 204 181 232 229 153 176 137 143 62
96 87 76 75 65 62 69 113 63 88 34 14 0 72 117 128 124 153 26 88 174 136 142 82
187 32 48 79 85 89 106 159 121 191 168 219 216 140 168 134 145 64
90 75 93 84 76 36 60 80 122 150 99 80 72 0 59 71 63 116 56 25 170 201 189 151
252 104 44 95 111 130 138 130 127 192 174 186 172 90 205 193 214 135
74 63 95 84 83 56 63 42 148 174 134 129 117 59 0 11 8 63 93 35 135 223 195 184
273 146 71 95 113 138 138 81 107 159 146 132 113 32 200 209 243 171
76 67 100 89 89 66 70 42 155 181 142 139 128 71 11 0 11 54 103 46 130 230 198
192 279 155 80 99 117 143 141 74 107 155 143 122 102 22 202 215 250 179
82 71 104 92 91 63 71 49 156 182 141 135 124 63 8 11 0 65 100 39 140 232 203 192
281 153 78 103 121 147 146 85 115 164 152 133 112 33 208 218 251 178
67 69 98 89 95 95 85 40 159 181 157 167 153 116 63 54 65 0 127 92 83 224 180 199
269 175 106 95 109 135 125 21 80 107 100 71 63 33 173 205 249 191
72 62 57 54 43 37 44 87 67 95 44 39 26 56 93 103 100 127 0 67 153 145 139 96 196
53 23 60 70 81 95 134 101 172 149 194 190 115 160 138 159 80
78 63 88 78 72 34 52 60 129 157 110 98 88 25 35 46 39 92 67 0 152 207 188 162
258 119 48 89 107 129 134 108 114 176 159 163 147 66 200 197 224 147
82 96 99 99 110 137 115 94 148 159 161 187 174 170 135 130 140 83 153 152 0 188

128 184 222 183 139 95 95 110 91 62 54 24 23 81 110 113 108 164 217 184
159 164 130 141 141 174 161 195 78 50 103 136 136 201 223 230 232 224 145 207
188 0 65 57 51 109 160 132 116 90 102 217 148 188 168 264 281 231 100 26 30 75
122 132 100 111 116 156 136 158 80 65 109 148 142 189 195 198 203 180 139 188
128 65 0 91 94 126 145 100 82 60 57 167 99 126 106 208 230 194 36 39 94 103
131 131 101 109 105 129 122 163 39 27 52 81 82 151 184 192 192 199 96 162 184 57
91 0 106 53 115 104 94 74 94 196 134 192 168 251 260 197 126 64 64 19
206 212 179 190 190 224 210 242 129 102 154 186 187 252 273 279 281 269 196 258
222 51 94 106 0 158 211 180 163 136 145 259 190 218 200 302 323 278 120 65 49
124
112 106 86 89 81 90 91 135 46 65 22 28 32 104 146 155 153 175 53 119 183 109 126
53 158 0 75 89 88 83 103 178 129 197 173 236 238 166 156 111 115 34
57 44 51 44 34 15 25 65 82 110 63 61 48 44 71 80 78 106 23 48 139 160 145 115
211 75 0 53 68 86 95 114 90 160 139 173 168 92 162 150 176 101
28 33 4 11 19 59 37 63 65 87 68 92 79 95 95 99 103 95 60 89 95 132 100 104 180
89 53 0 18 44 45 92 42 112 89 149 156 99 111 116 155 97
43 51 18 29 35 75 54 79 55 73 66 97 85 111 113 117 121 109 70 107 95 116 82 94
163 88 68 18 0 27 27 103 42 109 85 157 168 115 94 98 140 90
70 77 43 54 57 96 78 106 40 50 61 98 89 130 138 143 147 135 81 129 110 90 60 74
136 83 86 44 27 0 21 128 62 119 96 179 192 142 79 72 115 74
65 75 45 56 63 103 81 101 61 68 81 117 106 138 138 141 146 125 95 134 91 102 57
94 145 103 95 45 27 21 0 115 46 98 75 163 179 136 67 81 129 95
66 72 95 89 97 105 90 50 157 176 158 173 159 130 81 74 85 21 134 108 62 217 167
196 259 178 114 92 103 128 115 0 69 86 81 60 65 54 158 195 243 190
37 52 45 47 58 91 68 66 97 112 107 134 121 127 107 107 115 80 101 114 54 148 99
134 190 129 90 42 42 62 46 69 0 71 49 117 133 98 95 127 175 132
103 118 115 118 129 158 136 118 159 166 175 204 191 192 159 155 164 107 172 176
24 188 126 192 218 197 160 112 109 119 98 86 71 0 24 94 127 137 100 163 218 194
84 99 93 96 107 139 116 104 135 142 151 181 168 174 146 143 152 100 149 159 23
168 106 168 200 173 139 89 85 96 75 81 49 24 0 104 133 127 85 143 197 170
125 132 152 147 156 164 150 109 212 229 216 232 219 186 132 122 133 71 194 163
81 264 208 251 302 236 173 149 157 179 163 60 117 94 104 0 39 100 190 241 292
246
129 132 159 151 158 156 147 103 221 241 219 229 216 172 113 102 112 63 190 147
110 281 230 260 323 238 168 156 168 192 179 65 133 127 133 39 0 81 216 259 307
253
72 67 100 90 92 78 76 36 159 184 150 153 140 90 32 22 33 33 115 66 113 231 194
197 278 166 92 99 115 142 136 54 98 137 127 100 81 0 193 214 253 187
126 139 112 122 129 169 148 160 110 99 137 176 168 205 200 202 208 173 160 200
108 100 36 126 120 156 162 111 94 79 67 158 95 100 85 190 216 193 0 74 129 137
141 148 114 126 127 163 147 178 72 46 100 137 134 193 209 215 218 205 138 197
164 26 39 64 65 111 150 116 98 72 81 195 127 163 143 241 259 214 74 0 55 80
183 186 153 163 161 191 180 218 95 69 115 143 145 214 243 250 251 249 159 224
217 30 94 64 49 115 176 155 140 115 129 243 175 218 197 292 307 253 129 55 0 81
124 122 94 101 95 115 111 153 35 38 37 62 64 135 171 179 178 191 80 147 184 75
103 19 124 34 101 97 90 74 95 190 132 194 170 246 253 187 137 80 81 0
EOF
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,

22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41]
 [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (0, 8), (0, 9),
 (0, 10), (0, 11), (0, 12), (0, 13), (0, 14), (0, 15), (0, 16), (0, 17), (0, 18),
 (0, 19), (0, 20), (0, 21), (0, 22), (0, 23), (0, 24), (0, 25), (0, 26), (0, 27),
 (0, 28), (0, 29), (0, 30), (0, 31), (0, 32), (0, 33), (0, 34), (0, 35), (0, 36),
 (0, 37), (0, 38), (0, 39), (0, 40), (0, 41), (1, 0), (1, 1), (1, 2), (1, 3), (1,
 4), (1, 5), (1, 6), (1, 7), (1, 8), (1, 9), (1, 10), (1, 11), (1, 12), (1, 13),
 (1, 14), (1, 15), (1, 16), (1, 17), (1, 18), (1, 19), (1, 20), (1, 21), (1, 22),
 (1, 23), (1, 24), (1, 25), (1, 26), (1, 27), (1, 28), (1, 29), (1, 30), (1, 31),
 (1, 32), (1, 33), (1, 34), (1, 35), (1, 36), (1, 37), (1, 38), (1, 39), (1, 40),
 (1, 41), (2, 0), (2, 1), (2, 2), (2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8),
 (2, 9), (2, 10), (2, 11), (2, 12), (2, 13), (2, 14), (2, 15), (2, 16), (2, 17),
 (2, 18), (2, 19), (2, 20), (2, 21), (2, 22), (2, 23), (2, 24), (2, 25), (2, 26),
 (2, 27), (2, 28), (2, 29), (2, 30), (2, 31), (2, 32), (2, 33), (2, 34), (2, 35),
 (2, 36), (2, 37), (2, 38), (2, 39), (2, 40), (2, 41), (3, 0), (3, 1), (3, 2),
 (3, 3), (3, 4), (3, 5), (3, 6), (3, 7), (3, 8), (3, 9), (3, 10), (3, 11), (3,
 12), (3, 13), (3, 14), (3, 15), (3, 16), (3, 17), (3, 18), (3, 19), (3, 20), (3,
 21), (3, 22), (3, 23), (3, 24), (3, 25), (3, 26), (3, 27), (3, 28), (3, 29), (3,
 30), (3, 31), (3, 32), (3, 33), (3, 34), (3, 35), (3, 36), (3, 37), (3, 38), (3,
 39), (3, 40), (3, 41), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4), (4, 5), (4, 6),
 (4, 7), (4, 8), (4, 9), (4, 10), (4, 11), (4, 12), (4, 13), (4, 14), (4, 15),
 (4, 16), (4, 17), (4, 18), (4, 19), (4, 20), (4, 21), (4, 22), (4, 23), (4, 24),
 (4, 25), (4, 26), (4, 27), (4, 28), (4, 29), (4, 30), (4, 31), (4, 32), (4, 33),
 (4, 34), (4, 35), (4, 36), (4, 37), (4, 38), (4, 39), (4, 40), (4, 41), (5, 0),
 (5, 1), (5, 2), (5, 3), (5, 4), (5, 5), (5, 6), (5, 7), (5, 8), (5, 9), (5, 10),
 (5, 11), (5, 12), (5, 13), (5, 14), (5, 15), (5, 16), (5, 17), (5, 18), (5, 19),
 (5, 20), (5, 21), (5, 22), (5, 23), (5, 24), (5, 25), (5, 26), (5, 27), (5, 28),
 (5, 29), (5, 30), (5, 31), (5, 32), (5, 33), (5, 34), (5, 35), (5, 36), (5, 37),
 (5, 38), (5, 39), (5, 40), (5, 41), (6, 0), (6, 1), (6, 2), (6, 3), (6, 4), (6,
 5), (6, 6), (6, 7), (6, 8), (6, 9), (6, 10), (6, 11), (6, 12), (6, 13), (6, 14),
 (6, 15), (6, 16), (6, 17), (6, 18), (6, 19), (6, 20), (6, 21), (6, 22), (6, 23),
 (6, 24), (6, 25), (6, 26), (6, 27), (6, 28), (6, 29), (6, 30), (6, 31), (6, 32),
 (6, 33), (6, 34), (6, 35), (6, 36), (6, 37), (6, 38), (6, 39), (6, 40), (6, 41),
 (7, 0), (7, 1), (7, 2), (7, 3), (7, 4), (7, 5), (7, 6), (7, 7), (7, 8), (7, 9),
 (7, 10), (7, 11), (7, 12), (7, 13), (7, 14), (7, 15), (7, 16), (7, 17), (7, 18),
 (7, 19), (7, 20), (7, 21), (7, 22), (7, 23), (7, 24), (7, 25), (7, 26), (7, 27),
 (7, 28), (7, 29), (7, 30), (7, 31), (7, 32), (7, 33), (7, 34), (7, 35), (7, 36),
 (7, 37), (7, 38), (7, 39), (7, 40), (7, 41), (8, 0), (8, 1), (8, 2), (8, 3), (8,
 4), (8, 5), (8, 6), (8, 7), (8, 8), (8, 9), (8, 10), (8, 11), (8, 12), (8, 13),
 (8, 14), (8, 15), (8, 16), (8, 17), (8, 18), (8, 19), (8, 20), (8, 21), (8, 22),
 (8, 23), (8, 24), (8, 25), (8, 26), (8, 27), (8, 28), (8, 29), (8, 30), (8, 31),
 (8, 32), (8, 33), (8, 34), (8, 35), (8, 36), (8, 37), (8, 38), (8, 39), (8, 40),
 (8, 41), (9, 0), (9, 1), (9, 2), (9, 3), (9, 4), (9, 5), (9, 6), (9, 7), (9, 8),
 (9, 9), (9, 10), (9, 11), (9, 12), (9, 13), (9, 14), (9, 15), (9, 16), (9, 17),
 (9, 18), (9, 19), (9, 20), (9, 21), (9, 22), (9, 23), (9, 24), (9, 25), (9, 26),
 (9, 27), (9, 28), (9, 29), (9, 30), (9, 31), (9, 32), (9, 33), (9, 34), (9, 35),
 (9, 36), (9, 37), (9, 38), (9, 39), (9, 40), (9, 41), (10, 0), (10, 1), (10, 2),
 (10, 3), (10, 4), (10, 5), (10, 6), (10, 7), (10, 8), (10, 9), (10, 10), (10,

(25, 1), (25, 26), (25, 27), (25, 28), (25, 29), (25, 30), (25, 31), (25, 32), (25, 33), (25, 34), (25, 35), (25, 36), (25, 37), (25, 38), (25, 39), (25, 40), (25, 41), (26, 0), (26, 1), (26, 2), (26, 3), (26, 4), (26, 5), (26, 6), (26, 7), (26, 8), (26, 9), (26, 10), (26, 11), (26, 12), (26, 13), (26, 14), (26, 15), (26, 16), (26, 17), (26, 18), (26, 19), (26, 20), (26, 21), (26, 22), (26, 23), (26, 24), (26, 25), (26, 26), (26, 27), (26, 28), (26, 29), (26, 30), (26, 31), (26, 32), (26, 33), (26, 34), (26, 35), (26, 36), (26, 37), (26, 38), (26, 39), (26, 40), (26, 41), (27, 0), (27, 1), (27, 2), (27, 3), (27, 4), (27, 5), (27, 6), (27, 7), (27, 8), (27, 9), (27, 10), (27, 11), (27, 12), (27, 13), (27, 14), (27, 15), (27, 16), (27, 17), (27, 18), (27, 19), (27, 20), (27, 21), (27, 22), (27, 23), (27, 24), (27, 25), (27, 26), (27, 27), (27, 28), (27, 29), (27, 30), (27, 31), (27, 32), (27, 33), (27, 34), (27, 35), (27, 36), (27, 37), (27, 38), (27, 39), (27, 40), (27, 41), (28, 0), (28, 1), (28, 2), (28, 3), (28, 4), (28, 5), (28, 6), (28, 7), (28, 8), (28, 9), (28, 10), (28, 11), (28, 12), (28, 13), (28, 14), (28, 15), (28, 16), (28, 17), (28, 18), (28, 19), (28, 20), (28, 21), (28, 22), (28, 23), (28, 24), (28, 25), (28, 26), (28, 27), (28, 28), (28, 29), (28, 30), (28, 31), (28, 32), (28, 33), (28, 34), (28, 35), (28, 36), (28, 37), (28, 38),

[illegible]

```

11), (38, 12), (38, 13), (38, 14), (38, 15), (38, 16), (38, 17), (38, 18), (38,
19), (38, 20), (38, 21), (38, 22), (38, 23), (38, 24), (38, 25), (38, 26), (38,
27), (38, 28), (38, 29), (38, 30), (38, 31), (38, 32), (38, 33), (38, 34), (38,
35), (38, 36), (38, 37), (38, 38), (38, 39), (38, 40), (38, 41), (39, 0), (39,
1), (39, 2), (39, 3), (39, 4), (39, 5), (39, 6), (39, 7), (39, 8), (39, 9), (39,
10), (39, 11), (39, 12), (39, 13), (39, 14), (39, 15), (39, 16), (39, 17), (39,
18), (39, 19), (39, 20), (39, 21), (39, 22), (39, 23), (39, 24), (39, 25), (39,
26), (39, 27), (39, 28), (39, 29), (39, 30), (39, 31), (39, 32), (39, 33), (39,
34), (39, 35), (39, 36), (39, 37), (39, 38), (39, 39), (39, 40), (39, 41), (40,
0), (40, 1), (40, 2), (40, 3), (40, 4), (40, 5), (40, 6), (40, 7), (40, 8), (40,
9), (40, 10), (40, 11), (40, 12), (40, 13), (40, 14), (40, 15), (40, 16), (40,
17), (40, 18), (40, 19), (40, 20), (40, 21), (40, 22), (40, 23), (40, 24), (40,
25), (40, 26), (40, 27), (40, 28), (40, 29), (40, 30), (40, 31), (40, 32), (40,
33), (40, 34), (40, 35), (40, 36), (40, 37), (40, 38), (40, 39), (40, 40), (40,
41), (41, 0), (41, 1), (41, 2), (41, 3), (41, 4), (41, 5), (41, 6), (41, 7),
(41, 8), (41, 9), (41, 10), (41, 11), (41, 12), (41, 13), (41, 14), (41, 15),
(41, 16), (41, 17), (41, 18), (41, 19), (41, 20), (41, 21), (41, 22), (41, 23),
(41, 24), (41, 25), (41, 26), (41, 27), (41, 28), (41, 29), (41, 30), (41, 31),
(41, 32), (41, 33), (41, 34), (41, 35), (41, 36), (41, 37), (41, 38), (41, 39),
(41, 40), (41, 41)]

```

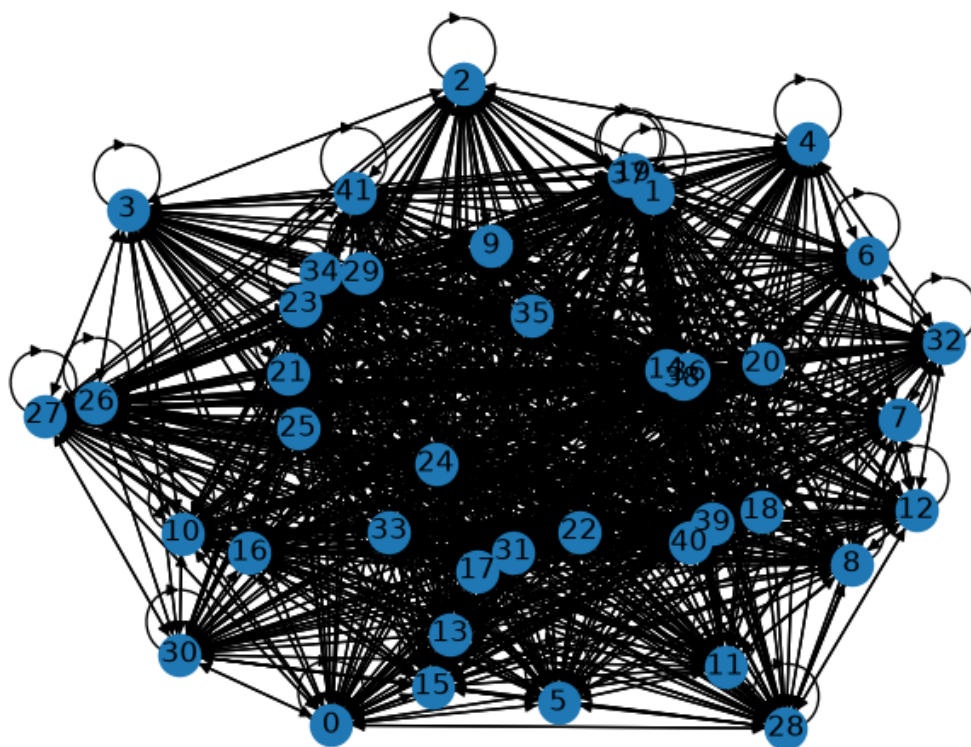
```

[4]: # Probamos algunas funciones del objeto problema, que ya nos vienen dadas
# Distancia entre nodos (ciudades)
distancia = problema.get_weight(3, 0)
print(distancia)

#Todas las funciones
#Documentación: https://tsplib95.readthedocs.io/en/v0.6.1/modules.html
#dir(problema)

import networkx as nx
import matplotlib.pyplot as plt
nx.draw(problema.get_graph(), with_labels=True, arrows=True)
plt.show()

```

```
[5]: import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.manifold import MDS # Multidimensional Scaling o Escalado
    ↪ Multidimensional

def plot_tsp_solution(distance_matrix, tsp_solution):
    """
    Dibuja el grafo de un TSP con las posiciones calculadas mediante MDS y
    ↪ muestra
    solo las aristas correspondientes a la solución del TSP.

    :param distance_matrix: np.ndarray, matriz de distancias entre nodos
    :param tsp_solution: list, lista de nodos en el orden de la solución del TSP
    """
    # Crear el grafo completo
    G = nx.Graph()
    num_nodes = len(distance_matrix)
    for i in range(num_nodes):
        for j in range(i + 1, num_nodes):
```

```

        G.add_edge(i, j, weight=distance_matrix[i][j])

    # Usar MDS para calcular posiciones de los nodos
    mds = MDS(n_components=2, metric="precomputed", init="random",
    ↪random_state=42)
    positions = mds.fit_transform(distance_matrix)

    # Convertir las posiciones en un diccionario para networkx
    pos = {i: positions[i] for i in range(num_nodes)}

    # Crear un subgrafo con las aristas del camino TSP
    TSP_G = nx.Graph()
    for i in range(len(tsp_solution) - 1):
        u = tsp_solution[i]
        v = tsp_solution[i + 1]
        TSP_G.add_edge(u, v, weight=distance_matrix[u][v])

    # Dibujar el grafo
    plt.figure(figsize=(8, 6))

    # Dibujar nodos
    nx.draw_networkx_nodes(G, pos, node_color='lightblue', node_size=500)

    # Dibujar las aristas del camino TSP
    nx.draw_networkx_edges(TSP_G, pos, edge_color='red', width=2)

    # Añadir etiquetas a los nodos y pesos de las aristas
    nx.draw_networkx_labels(G, pos, font_size=10, font_weight='bold')
    edge_labels = nx.get_edge_attributes(TSP_G, 'weight')
    nx.draw_networkx_edge_labels(TSP_G, pos, edge_labels=edge_labels,
    ↪font_size=8)

    plt.title("Grafo TSP con solución específica")
    plt.show()

```

2.1.1 Funciones básicas

```

[9]: # Genera una solución aleatoria con comienzo en el nodo 0
def crear_solucion(nodos):
    # Nodo inicial = 0
    solucion = [nodos[0]]
    # Para el resto de nodos, los recorremos e incluimos aleatoriamente en la
    ↪solución cada uno de ellos
    for n in nodos[1:]:
        # Debemos quitar el nodo 0 y los ya incluidos en cada iteración
        solucion = solucion + [random.choice(list(set(nodos) - set({nodos[0]}))
    ↪- set(solucion)))]

```

```

    return solucion

# Devuelve la distancia entre dos nodos aprovechando la función get_weight() de
↳ tsplib95
def distancia(a,b, problema):
    return problema.get_weight(a,b)

# Devuelve la distancia total de una trayectoria/solución
def distancia_total(solucion, problema):
    distancia_total = 0
    # Para cada nodo de la solución calculamos su distancia a siguiente nodo
    for i in range(len(solucion)-1):
        distancia_total += distancia(solucion[i] ,solucion[i+1] , problema)
    # Devolvemos la distancia total más la distancia del último nodo al primero
↳ para cerrar viaje
    return distancia_total + distancia(solucion[len(solucion)-1] ,solucion[0],
↳ problema)

solucion = crear_solucion(nodos)
print(solucion)
print(distancia_total(solucion, problema))

```

```

[0, 10, 40, 8, 37, 13, 19, 25, 39, 41, 32, 12, 22, 2, 14, 21, 24, 9, 16, 15, 28,
36, 17, 26, 23, 34, 6, 35, 31, 20, 11, 3, 27, 4, 30, 5, 18, 33, 29, 38, 7, 1]
4256

```

2.2 Búsqueda aleatoria - TSP

- El algoritmo inicializa soluciones al azar y se queda con la mejor que encuentre
- Se realizan N iteraciones y nos quedamos con la mejor
- Todo es azar, sin que garanticemos solución óptima

```

[17]: #####
# BUSQUEDA ALEATORIA
#####

def busqueda_aleatoria(problema, iteraciones):
    # Obtenemos listado de nodos del problema
    nodos = list(problema.get_nodos())

    mejor_solucion = []
    mejor_distancia_total = float("inf")

    for i in range(iteraciones):
        # Generamos solución aleatoria

```

```

solucion = crear_solucion(nodos)
# Calculamos distancia total de la solución
distancia = distancia_total(solucion, problema)
if (distancia < mejor_distancia_total):
    mejor_solucion = solucion
    mejor_distancia_total = distancia

return mejor_solucion, mejor_distancia_total

iteraciones = 5000
mejor_solucion, mejor_distancia_total = busqueda_aleatoria(problema,
    iteraciones)

print(f"Solución: {mejor_solucion}")
print(f"Distancia: {mejor_distancia_total}")

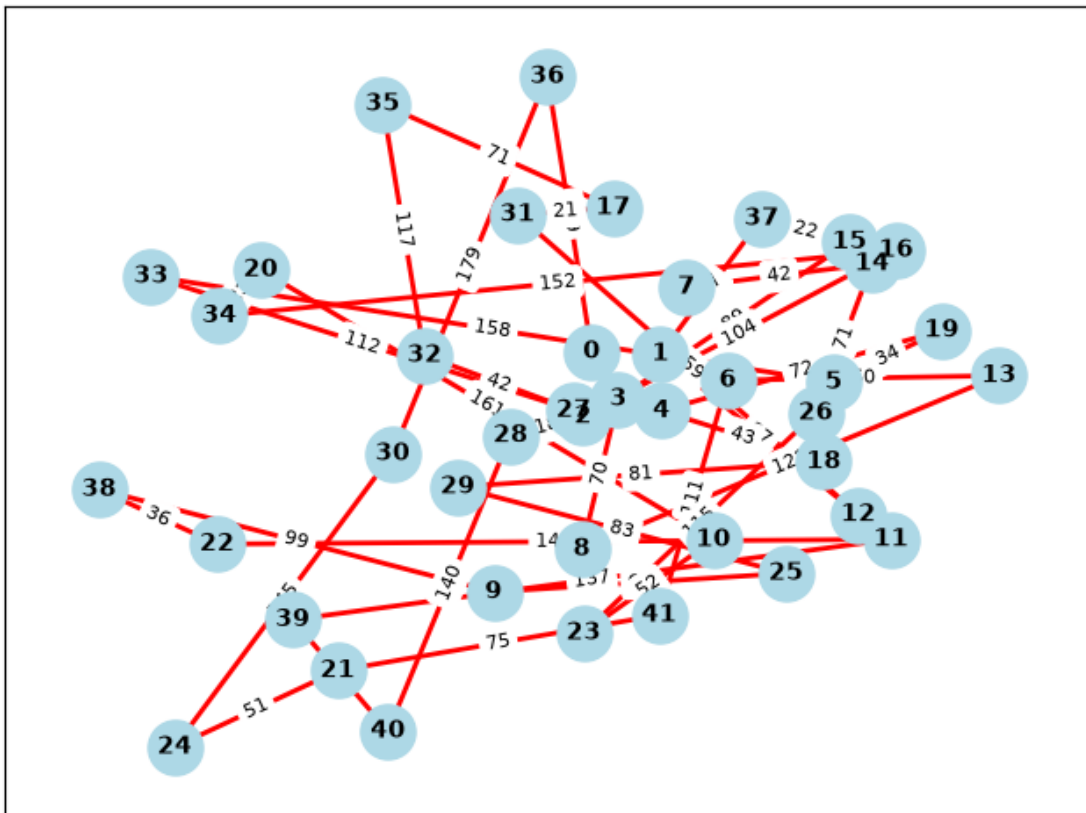
plot_tsp_solution(problema.edge_weights, mejor_solucion)

```

Solución: [0, 36, 30, 24, 21, 41, 6, 13, 8, 3, 15, 37, 1, 12, 31, 17, 35, 32, 27, 33, 5, 19, 4, 18, 29, 25, 9, 38, 22, 11, 39, 40, 28, 2, 16, 34, 20, 10, 23, 26, 14, 7]

Distancia: 3655

Grafo TSP con solución específica



2.3 Búsqueda local - TSP

- El algoritmo se va quedando con vecinos (operadores de vecindad) para intentar converger a mejores soluciones
- Se realizan N iteraciones y nos quedamos con la mejor

```
[33]: # Función que a partir de una solución genera la vecina óptima (es decir, la
      ↪ mejor vecina de todas las que existen)
def genera_mejor_vecina(solucion):
    # Generador de soluciones vecinas: 2-opt (intercambiar 2 nodos) Si hay N
    ↪ nodos se generan (N-1)x(N-2)/2 soluciones en el bucle
    # Se puede modificar para aplicar otros generadores distintos que 2-opt
    ↪ como inversión, barajado...
    #print(solucion)

    mejor_solucion = []
    mejor_distancia = float("inf")

    #Recorremos todos los nodos en bucle doble para evaluar todos los
    ↪ intercambios 2-opt
    for i in range(1,len(solucion)-1):
        for j in range(i+1, len(solucion)):
            # Generamos nueva solución vecina (0, 1, 2, 3, i, ..., j, ..., 41)
            ↪ -> (0, 1, 2, 3, j, ..., i, ..., 41)
            vecina = solucion[:i] + [solucion[j]] + solucion[i+1:j] +
            ↪ [solucion[i]] + solucion[j+1:]

            # Se evalua la nueva solución
            distancia_vecina = distancia_total(vecina, problema)

            # Si es la mejor solución, la almacenamos
            if (distancia_vecina <= mejor_distancia):
                mejor_distancia = distancia_vecina
                mejor_solucion = vecina

    # Devolvemos mejor vecina
    return mejor_solucion

solucion = crear_solucion(nodos)
print(f"Distancia Solucion Incial: {distancia_total(solucion, problema)}")

nueva_solucion = genera_mejor_vecina(solucion)
```

```
print(f"Distancia Mejor Solucion Local: {distancia_total(nueva_solucion,
↪problema)}")
```

Distancia Solucion Inicial: 4491

Distancia Mejor Solucion Local: 4035

```
[37]: # En este caso, la búsqueda se ejecuta sin criterio de parada, se para cuando
↪no es posible mejorar.
# Aquí iremos recorriendo las mejores vecinas para ir construyendo trayectoria
def busqueda_local(problema):
    mejor_solucion = []

    # Generar una solucion inicial de referencia(aleatoria)
    solucion_referencia = crear_solucion(nodos)
    mejor_distancia = distancia_total(solucion_referencia, problema)

    # Contador de iteraciones (por tener referencia)
    iteracion = 0

    while(1):
        iteracion += 1
        #print('#',iteracion)

        # Obtenemos la mejor vecina
        vecina = genera_mejor_vecina(solucion_referencia)

        # Evaluamos para ver si mejoramos respecto a lo encontrado hasta el
↪momento
        distancia_vecina = distancia_total(vecina, problema)

        # Si mejoramos hay que seguir
        if (distancia_vecina < mejor_distancia):
            #mejor_solucion = copy.deepcopy(vecina) # Con copia profunda. Las
↪copias en python son por referencia
            mejor_solucion = vecina # Guarda la mejor
↪solución encontrada
            mejor_distancia = distancia_vecina

        # Si no mejoramos, retornamos ya que hemos llegado a un minimo local
↪(según nuestro operador de vecindad 2-opt)
        else:
            #print(f"En la iteracion {iteracion} la mejor solución encontrada
↪es: {mejor_solucion}")
            #print(f"Distancia: {mejor_distancia}")
            return mejor_solucion, mejor_distancia, iteracion

    solucion_referencia = vecina
```

```

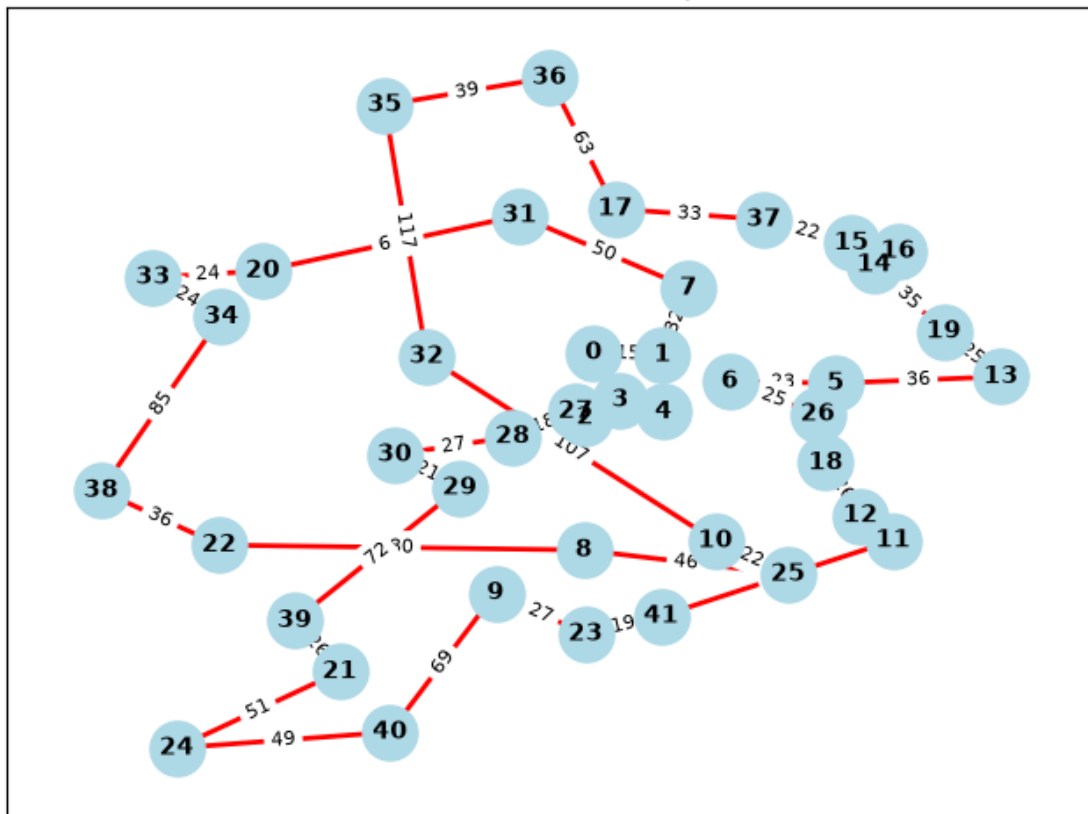
mejor_solucion, mejor_distancia, iteracion = busqueda_local(problema)
print(f"En la iteracion {iteracion} la mejor solución encontrada es:␣
↪{mejor_solucion}")
print(f"Distancia: {mejor_distancia}")

plot_tsp_solution(problema.edge_weights, mejor_solucion)

```

En la iteracion 43 la mejor solución encontrada es: [0, 1, 7, 31, 20, 33, 34, 38, 22, 8, 25, 10, 32, 35, 36, 17, 37, 15, 16, 14, 19, 13, 5, 6, 26, 18, 12, 11, 41, 23, 9, 40, 24, 21, 39, 29, 30, 28, 27, 2, 4, 3]
Distancia: 1580

Grafo TSP con solución específica



2.4 Recocido simulado (Simulated Annealing) - TSP

- El algoritmo se va quedando con vecinos (operadores de vecindad) para intentar converger a mejores soluciones, aunque ahora a veces puedo elegir vecinos peores
- Al principio, elijo más veces vecinos peores (diversifico) y luego elijo menos veces esos vecinos peores (intensifico)

- Se realizan N iteraciones o bajamos temperatura al máximo

2.4.1 Funciones auxiliares

```
[55]: #####
# SIMULATED ANNEALING
#####

# Generador de 1 solución vecina 2-opt 100% aleatoria (intercambiar 2 nodos)
def genera_vecina_aleatorio(solucion):

    # Se eligen dos nodos aleatoriamente
    i,j = sorted(random.sample(range(1,len(solucion)) , 2))

    # Devuelve una nueva solución pero intercambiando los dos nodos elegidos al
    ↪azar
    return solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] +
    ↪solucion[j+1:]

# Funcion de probabilidad para aceptar peores soluciones
def probabilidad(T, d):
    if random.random() < math.exp(-1*d / T) :
        return True
    else:
        return False

# Funcion de descenso de temperatura
def bajar_temperatura(T):
    return T*0.9999
```

```
[56]: def recocado_simulado(problema, temperatura):
    # Generamos solución de referencia
    solucion_referencia = crear_solucion(nodos)
    distancia_referencia = distancia_total(solucion_referencia, problema)

    mejor_solucion = []
    mejor_distancia = float("inf")

    iteraciones = 0

    # Mientras la temperatura no sea 0 (aquí no hay parada por iteraciones)
    while temperatura > .0001:
        iteraciones += 1

        # Genera una solución vecina
```



```

vecina = genera_vecina_aleatorio(solucion_referencia)

# Calculamos distancia
distancia_vecina = distancia_total(vecina, problema)

# Si es la mejor solución de todas se guarda siempre
if (distancia_vecina < mejor_distancia):
    mejor_solucion = vecina
    mejor_distancia = distancia_vecina

# Si la nueva vecina es o
# Si el vecino es peor se cambia según una probabilidad que depende de  $T$  y  $\Delta T$ 
if ((distancia_vecina < distancia_referencia) or
    probabilidad(temperatura, abs(distancia_referencia - distancia_vecina))):
    #solucion_referencia = copy.deepcopy(vecina)
    solucion_referencia = vecina
    distancia_referencia = distancia_vecina

# Bajamos la temperatura
temperatura = bajar_temperatura(temperatura)

return mejor_solucion, mejor_distancia, iteraciones

temperatura = 100000
mejor_solucion, mejor_distancia, iteraciones = recocido_simulado(problema,
    temperatura)

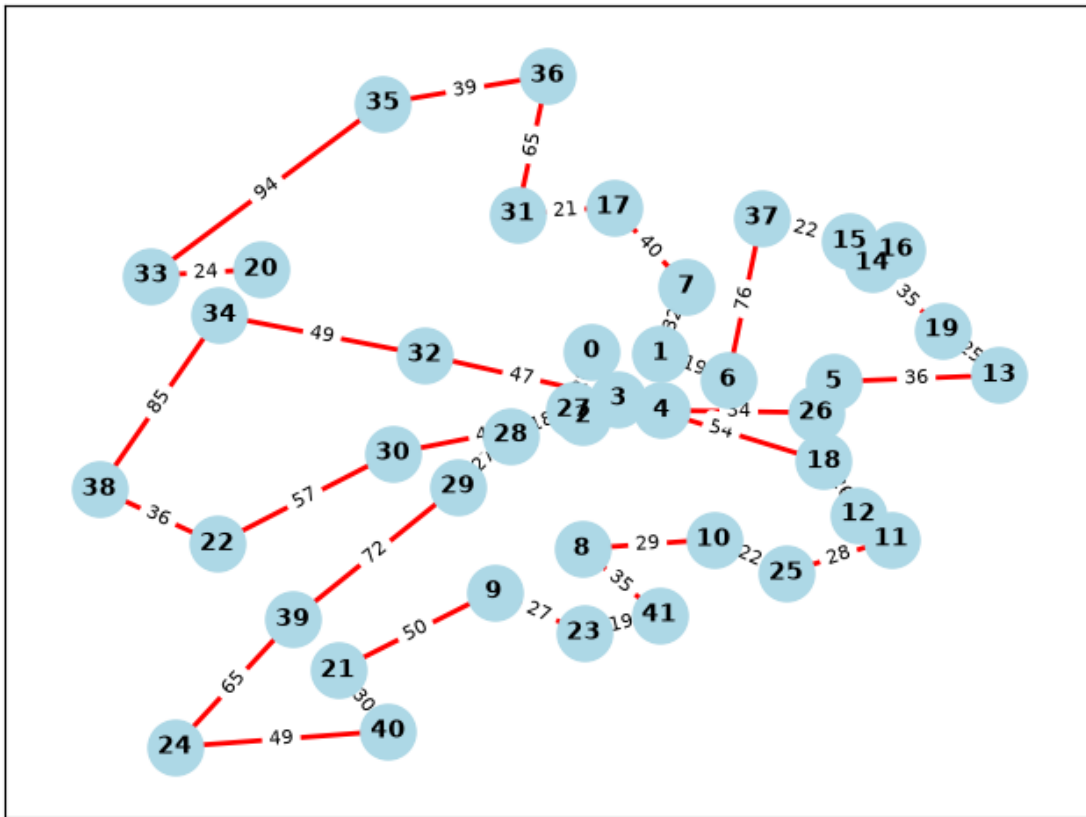
print(f"La mejor solución encontrada es: {mejor_solucion} en {iteraciones} iteraciones")
print(f"Distancia: {mejor_distancia}")

plot_tsp_solution(problema.edge_weights, mejor_solucion)

```

La mejor solución encontrada es: [0, 27, 28, 29, 39, 24, 40, 21, 9, 23, 41, 8, 10, 25, 11, 12, 18, 3, 32, 34, 38, 22, 30, 2, 4, 26, 5, 13, 19, 14, 16, 15, 37, 6, 1, 7, 17, 31, 36, 35, 33, 20] en 207223 iteraciones
 Distancia: 1608

Grafo TSP con solución específica



3 Actividades Extra

3.1 A) Mejora de algoritmo de búsqueda local

3.1.1 1) Búsqueda local con multiarranque

La búsqueda local puede quedar atrapada en un mínimo local que depende de la solución inicial. El **multiarranque** ejecuta varias veces la búsqueda local desde soluciones iniciales aleatorias distintas y conserva la mejor solución obtenida entre todas las ejecuciones.

```
[ ]: #####
# BÚSQUEDA LOCAL DESDE UNA SOLUCIÓN INICIAL
#####

def busqueda_local_desde_solucion(solucion_inicial, problema):
    """
    Ejecuta la búsqueda local ya utilizada en el notebook, pero comenzando
    desde una solución inicial recibida como parámetro.

    Retorna:
```

```

        mejor_solucion: mejor recorrido encontrado.
        mejor_distancia: distancia total del recorrido.
        iteraciones: número de iteraciones realizadas.
    """
    solucion_referencia = solucion_inicial.copy()
    mejor_solucion = solucion_referencia.copy()
    mejor_distancia = distancia_total(mejor_solucion, problema)
    iteraciones = 0

    while True:
        iteraciones += 1

        # Se utiliza la función de vecindad definida anteriormente.
        vecina = genera_mejor_vecina(solucion_referencia)
        distancia_vecina = distancia_total(vecina, problema)

        if distancia_vecina < mejor_distancia:
            mejor_solucion = vecina.copy()
            mejor_distancia = distancia_vecina
            solucion_referencia = vecina
        else:
            # No existe mejora en este entorno: se alcanzó un mínimo local.
            break

    return mejor_solucion, mejor_distancia, iteraciones

#####
# MULTIARRANQUE
#####

def busqueda_local_multiarranque(problema, numero_arranques):
    """
    Ejecuta la búsqueda local desde varias soluciones iniciales aleatorias
    y conserva el mejor resultado global.

    Retorna:
        mejor_solucion_global: mejor recorrido de todos los arranques.
        mejor_distancia_global: distancia de dicho recorrido.
        resultados: información de cada arranque.
    """
    if numero_arranques <= 0:
        raise ValueError("El número de arranques debe ser mayor que cero.")

    nodos_problema = list(problema.get_nodes())
    mejor_solucion_global = None
    mejor_distancia_global = float("inf")

```

```

resultados = []

for arranque in range(1, numero_arranques + 1):
    solucion_inicial = crear_solucion(nodos_problema)
    distancia_inicial = distancia_total(solucion_inicial, problema)

    solucion_local, distancia_local, iteraciones = \
        busqueda_local_desde_solucion(solucion_inicial, problema)

    resultados.append({
        "arranque": arranque,
        "distancia_inicial": distancia_inicial,
        "distancia_final": distancia_local,
        "iteraciones": iteraciones
    })

    if distancia_local < mejor_distancia_global:
        mejor_solucion_global = solucion_local.copy()
        mejor_distancia_global = distancia_local

return mejor_solucion_global, mejor_distancia_global, resultados

```

```

[41]: # Ejemplo de ejecución del multiarranque
numero_arranques = 10

mejor_solucion_ma, mejor_distancia_ma, resultados_ma = \
    busqueda_local_multiarranque(problema, numero_arranques)

for resultado in resultados_ma:
    print(
        f"Arranque {resultado['arranque']:2d}: "
        f"distancia inicial = {resultado['distancia_inicial']:4d}, "
        f"distancia final = {resultado['distancia_final']:4d}, "
        f"iteraciones = {resultado['iteraciones']}"
    )

print("\nMejor solución obtenida mediante multiarranque:")
print(mejor_solucion_ma)
print(f"Mejor distancia: {mejor_distancia_ma}")

plot_tsp_solution(problema.edge_weights, mejor_solucion_ma)

```

```

Arranque  1: distancia inicial = 5335, distancia final = 1955, iteraciones = 37
Arranque  2: distancia inicial = 4114, distancia final = 1921, iteraciones = 33
Arranque  3: distancia inicial = 4965, distancia final = 1706, iteraciones = 39
Arranque  4: distancia inicial = 4751, distancia final = 1678, iteraciones = 36
Arranque  5: distancia inicial = 5077, distancia final = 1864, iteraciones = 29
Arranque  6: distancia inicial = 4915, distancia final = 1744, iteraciones = 46

```

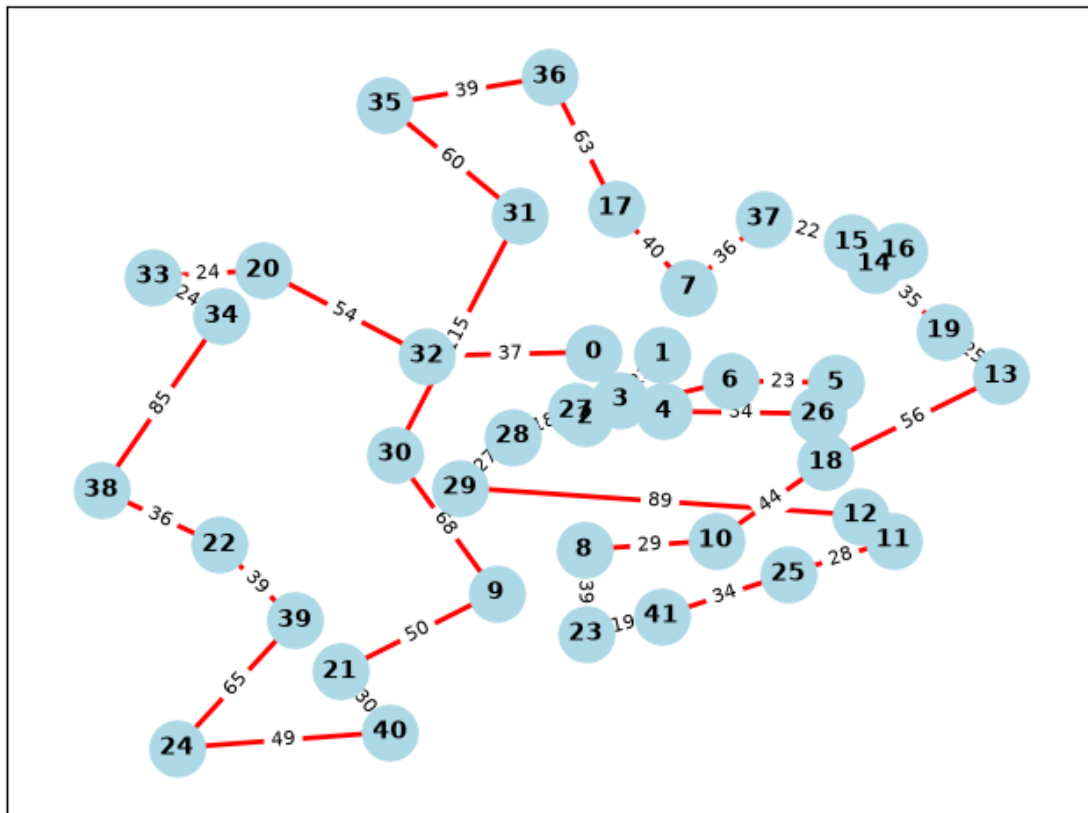
Arranque 7: distancia inicial = 4987, distancia final = 1573, iteraciones = 40
 Arranque 8: distancia inicial = 4601, distancia final = 1748, iteraciones = 37
 Arranque 9: distancia inicial = 4925, distancia final = 1935, iteraciones = 41
 Arranque 10: distancia inicial = 5302, distancia final = 1788, iteraciones = 32

Mejor solución obtenida mediante multiarranque:

[0, 32, 20, 33, 34, 38, 22, 39, 24, 40, 21, 9, 30, 31, 35, 36, 17, 7, 37, 15, 16, 14, 19, 13, 18, 10, 8, 23, 41, 25, 11, 12, 29, 28, 27, 2, 6, 5, 26, 4, 3, 1]

Mejor distancia: 1573

Grafo TSP con solución específica



La estrategia de multiarranque consistió en ejecutar la búsqueda local diez veces, utilizando en cada ejecución una solución inicial generada aleatoriamente. Como resultado, se obtuvieron soluciones finales con distintas calidades, cuyas distancias oscilaron entre 1573 y 1955. La mejor solución encontrada presentó una distancia de 1573, mejorando significativamente todas las soluciones iniciales, que se encontraban entre 4114 y 5335. Estos resultados evidencian que el desempeño de la búsqueda local depende de la solución desde la cual comienza el proceso, ya que diferentes puntos de partida pueden conducir a distintos óptimos locales. En consecuencia, la estrategia de multiarranque incrementa la probabilidad de encontrar soluciones de mejor calidad al explorar diversas regiones del espacio de búsqueda y seleccionar finalmente la mejor de ellas.

3.1.2 2) Búsqueda en entornos variables

La búsqueda en entornos variables evita depender de un único operador de vecindad. En este caso se emplean tres entornos:

1. **Intercambio:** intercambia las posiciones de dos nodos.
2. **Inversión:** invierte el segmento comprendido entre dos posiciones.
3. **Inserción:** extrae un nodo y lo inserta en otra posición.

Cuando un entorno encuentra una mejora, la búsqueda vuelve al primer entorno. Cuando ninguno de los tres entornos mejora la solución actual, el algoritmo termina.

```
[ ]: #####
# OPERADORES PARA LOS DISTINTOS ENTORNOS
#####

def mejor_vecina_intercambio(solucion, problema):
    """Obtiene la mejor vecina intercambiando dos nodos, sin mover el nodo 0."""
    mejor_vecina = solucion.copy()
    mejor_distancia = distancia_total(solucion, problema)

    for i in range(1, len(solucion) - 1):
        for j in range(i + 1, len(solucion)):
            vecina = solucion.copy()
            vecina[i], vecina[j] = vecina[j], vecina[i]
            distancia_vecina = distancia_total(vecina, problema)

            if distancia_vecina < mejor_distancia:
                mejor_vecina = vecina
                mejor_distancia = distancia_vecina

    return mejor_vecina, mejor_distancia

def mejor_vecina_inversion(solucion, problema):
    """Obtiene la mejor vecina invirtiendo un segmento del recorrido."""
    mejor_vecina = solucion.copy()
    mejor_distancia = distancia_total(solucion, problema)

    for i in range(1, len(solucion) - 1):
        for j in range(i + 1, len(solucion)):
            vecina = solucion[:i] + solucion[i:j + 1][::-1] + solucion[j + 1:]
            distancia_vecina = distancia_total(vecina, problema)

            if distancia_vecina < mejor_distancia:
                mejor_vecina = vecina
                mejor_distancia = distancia_vecina

    return mejor_vecina, mejor_distancia
```

```

def mejor_vecina_insercion(solucion, problema):
    """Obtiene la mejor vecina trasladando un nodo a otra posición."""
    mejor_vecina = solucion.copy()
    mejor_distancia = distancia_total(solucion, problema)

    for i in range(1, len(solucion)):
        for j in range(1, len(solucion)):
            if i == j:
                continue

            vecina = solucion.copy()
            nodo = vecina.pop(i)
            vecina.insert(j, nodo)
            distancia_vecina = distancia_total(vecina, problema)

            if distancia_vecina < mejor_distancia:
                mejor_vecina = vecina
                mejor_distancia = distancia_vecina

    return mejor_vecina, mejor_distancia

```

```

[44]: #####
# BÚSQUEDA EN ENTORNOS VARIABLES
#####

def busqueda_entornos_variables(problema, solucion_inicial=None):
    """
    Aplica una búsqueda descendente en entornos variables.

    Si un entorno mejora la solución, se vuelve al primer entorno.
    Si no mejora, se prueba el entorno siguiente.

    Retorna:
    mejor_solucion: mejor recorrido encontrado.
    mejor_distancia: distancia total del recorrido.
    iteraciones: cantidad de entornos evaluados.
    historial: mejoras producidas durante la búsqueda.
    """
    nodos_problema = list(problema.get_nodos())

    if solucion_inicial is None:
        solucion_actual = crear_solucion(nodos_problema)
    else:
        solucion_actual = solucion_inicial.copy()

```

```

distancia_actual = distancia_total(solucion_actual, problema)

entornos = [
    ("intercambio", mejor_vecina_intercambio),
    ("inversión", mejor_vecina_inversion),
    ("inserción", mejor_vecina_insercion)
]

k = 0
iteraciones = 0
historial = []

while k < len(entornos):
    iteraciones += 1
    nombre_entorno, funcion_entorno = entornos[k]

    vecina, distancia_vecina = funcion_entorno(solucion_actual, problema)

    if distancia_vecina < distancia_actual:
        solucion_actual = vecina.copy()
        distancia_actual = distancia_vecina
        historial.append({
            "iteracion": iteraciones,
            "entorno": nombre_entorno,
            "distancia": distancia_actual
        })

        # Al mejorar, se vuelve a explorar desde el primer entorno.
        k = 0
    else:
        # Si no mejora, se cambia al entorno siguiente.
        k += 1

return solucion_actual, distancia_actual, iteraciones, historial

```

```

[45]: # Ejemplo de ejecución de la búsqueda en entornos variables
solucion_inicial_bev = crear_solucion(nodos)
distancia_inicial_bev = distancia_total(solucion_inicial_bev, problema)

mejor_solucion_bev, mejor_distancia_bev, iteraciones_bev, historial_bev = \
    busqueda_entornos_variables(problema, solucion_inicial_bev)

print(f"Distancia inicial: {distancia_inicial_bev}")
print(f"Distancia final: {mejor_distancia_bev}")
print(f"Entornos evaluados: {iteraciones_bev}")
print(f"Cantidad de mejoras: {len(historial_bev)}")

```



```

print("\nMejoras encontradas:")
for mejora in historial_bev:
    print(
        f"Iteración {mejora['iteracion']:2d} - "
        f"entorno: {mejora['entorno']:11s} - "
        f"distancia: {mejora['distancia']}"
    )

print("\nMejor solución obtenida mediante entornos variables:")
print(mejor_solucion_bev)

plot_tsp_solution(problema.edge_weights, mejor_solucion_bev)

```

Distancia inicial: 4352

Distancia final: 1273

Entornos evaluados: 61

Cantidad de mejoras: 48

Mejoras encontradas:

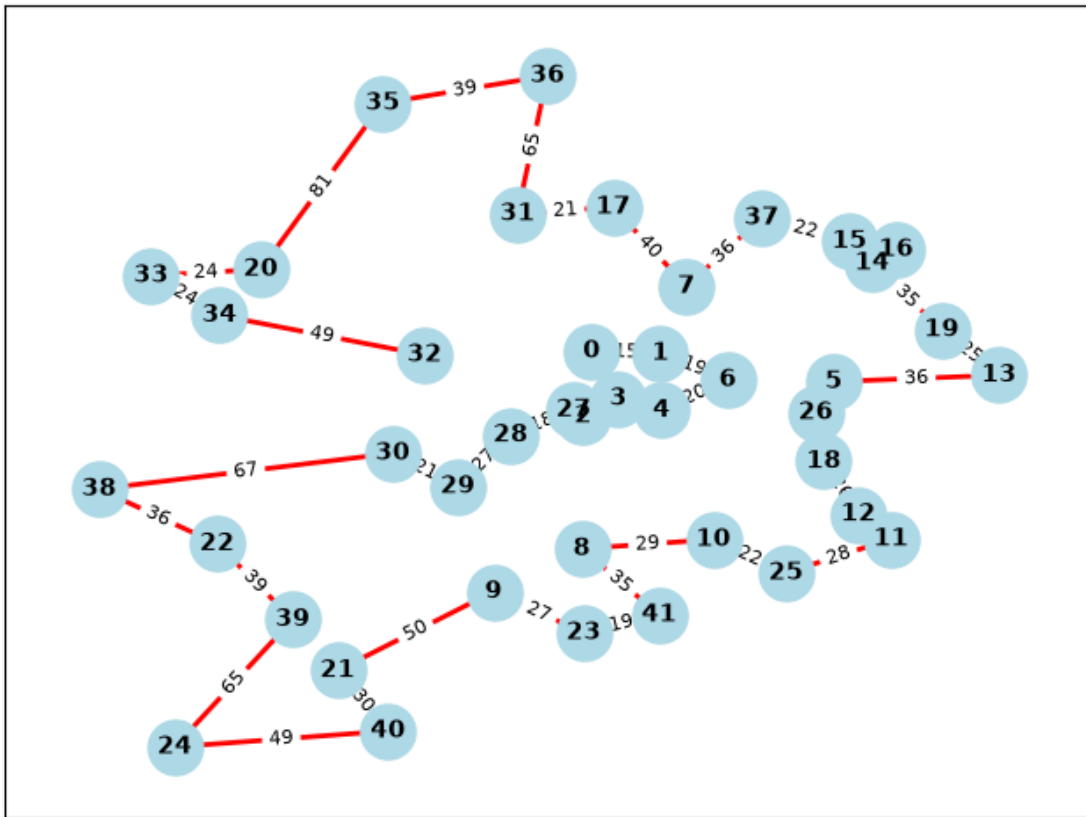
Iteración	1	-	entorno: intercambio	-	distancia: 3848
Iteración	2	-	entorno: intercambio	-	distancia: 3487
Iteración	3	-	entorno: intercambio	-	distancia: 3189
Iteración	4	-	entorno: intercambio	-	distancia: 2948
Iteración	5	-	entorno: intercambio	-	distancia: 2718
Iteración	6	-	entorno: intercambio	-	distancia: 2558
Iteración	7	-	entorno: intercambio	-	distancia: 2407
Iteración	8	-	entorno: intercambio	-	distancia: 2273
Iteración	9	-	entorno: intercambio	-	distancia: 2204
Iteración	10	-	entorno: intercambio	-	distancia: 2141
Iteración	11	-	entorno: intercambio	-	distancia: 2099
Iteración	12	-	entorno: intercambio	-	distancia: 2062
Iteración	13	-	entorno: intercambio	-	distancia: 2015
Iteración	14	-	entorno: intercambio	-	distancia: 1987
Iteración	15	-	entorno: intercambio	-	distancia: 1961
Iteración	16	-	entorno: intercambio	-	distancia: 1938
Iteración	17	-	entorno: intercambio	-	distancia: 1919
Iteración	18	-	entorno: intercambio	-	distancia: 1896
Iteración	19	-	entorno: intercambio	-	distancia: 1877
Iteración	20	-	entorno: intercambio	-	distancia: 1859
Iteración	21	-	entorno: intercambio	-	distancia: 1820
Iteración	22	-	entorno: intercambio	-	distancia: 1804
Iteración	23	-	entorno: intercambio	-	distancia: 1789
Iteración	24	-	entorno: intercambio	-	distancia: 1776
Iteración	25	-	entorno: intercambio	-	distancia: 1764
Iteración	26	-	entorno: intercambio	-	distancia: 1753
Iteración	27	-	entorno: intercambio	-	distancia: 1740
Iteración	28	-	entorno: intercambio	-	distancia: 1724
Iteración	29	-	entorno: intercambio	-	distancia: 1718

Iteración 30 - entorno: intercambio - distancia: 1694
Iteración 31 - entorno: intercambio - distancia: 1690
Iteración 33 - entorno: inversión - distancia: 1628
Iteración 34 - entorno: intercambio - distancia: 1625
Iteración 35 - entorno: intercambio - distancia: 1614
Iteración 37 - entorno: inversión - distancia: 1536
Iteración 38 - entorno: intercambio - distancia: 1522
Iteración 39 - entorno: intercambio - distancia: 1516
Iteración 40 - entorno: intercambio - distancia: 1512
Iteración 42 - entorno: inversión - distancia: 1482
Iteración 44 - entorno: inversión - distancia: 1457
Iteración 45 - entorno: intercambio - distancia: 1443
Iteración 47 - entorno: inversión - distancia: 1362
Iteración 49 - entorno: inversión - distancia: 1337
Iteración 50 - entorno: intercambio - distancia: 1333
Iteración 51 - entorno: intercambio - distancia: 1329
Iteración 53 - entorno: inversión - distancia: 1314
Iteración 55 - entorno: inversión - distancia: 1284
Iteración 58 - entorno: inserción - distancia: 1273

Mejor solución obtenida mediante entornos variables:

[0, 1, 6, 4, 3, 2, 27, 28, 29, 30, 38, 22, 39, 24, 40, 21, 9, 23, 41, 8, 10, 25,
11, 12, 18, 26, 5, 13, 19, 14, 16, 15, 37, 7, 17, 31, 36, 35, 20, 33, 34, 32]

Grafo TSP con solución específica



La búsqueda en entornos variables obtuvo una reducción significativa de la distancia total de la ruta, pasando de una solución inicial con distancia 4352 a una solución final con distancia 1273. Esto representa una mejora aproximada del 70,7 %. Durante el proceso se evaluaron 61 entornos y se encontraron 48 movimientos de mejora. Inicialmente, la mayor parte de las reducciones se consiguió mediante el entorno de intercambio de nodos. Posteriormente, al no encontrarse nuevas mejoras con dicho entorno, el algoritmo utilizó otros tipos de movimientos, como la inserción, alcanzando una solución de mejor calidad. Este comportamiento evidencia la ventaja de la búsqueda en entornos variables frente a una búsqueda local convencional, ya que permite cambiar la estructura de vecindad y disminuir la posibilidad de quedar detenido prematuramente en un óptimo local.

3.1.3 Interpretación de los resultados

El **multiarranque** mejora la búsqueda local porque explora varias regiones del espacio de soluciones. Cada ejecución puede terminar en un mínimo local distinto y, al conservar el mejor resultado, disminuye la dependencia de una única solución inicial aleatoria.

La **búsqueda en entornos variables** amplía la exploración al utilizar diferentes operadores de vecindad. Una solución que no puede mejorar mediante intercambio todavía puede mejorar mediante inversión o inserción. Por ello, esta estrategia tiene mayores posibilidades de escapar de mínimos locales asociados a un único entorno.

3.2 B) Recocido Simulado

3.2.1 Mejora en la selección de vecinos

En la versión original se genera un único vecino aleatorio mediante el intercambio de dos nodos. En esta mejora se generan varios candidatos utilizando tres operadores de vecindad: **intercambio**, **inversión** e **inserción**. Posteriormente se selecciona el candidato con menor distancia y se aplica la misma regla de aceptación del recocido simulado. De este modo se mejora la calidad del vecino evaluado sin modificar las funciones dadas inicialmente en la actividad.

```
[57]: #####
# MEJORA EN LA SELECCIÓN DE VECINOS PARA RECOCIDO SIMULADO
#####

# Genera un vecino intercambiando dos nodos.
# Se mantiene fijo el nodo inicial ubicado en la posición 0.
def vecino_intercambio_rs(solucion):
    vecina = solucion.copy()
    i, j = random.sample(range(1, len(vecina)), 2)
    vecina[i], vecina[j] = vecina[j], vecina[i]
    return vecina

# Genera un vecino invirtiendo el segmento comprendido entre dos posiciones.
def vecino_inversion_rs(solucion):
    i, j = sorted(random.sample(range(1, len(solucion)), 2))
    return solucion[:i] + solucion[i:j + 1][::-1] + solucion[j + 1:]

# Genera un vecino extrayendo un nodo e insertándolo en otra posición.
def vecino_insercion_rs(solucion):
    vecina = solucion.copy()
    i, j = random.sample(range(1, len(vecina)), 2)
    nodo = vecina.pop(i)
    vecina.insert(j, nodo)
    return vecina

def seleccionar_vecino_mejorado(solucion, problema, cantidad_candidatos=6):
    """
    Genera varios vecinos con diferentes operadores y devuelve el candidato
    con menor distancia. También informa qué operador produjo dicho vecino.
    """
    if cantidad_candidatos < 1:
        raise ValueError("cantidad_candidatos debe ser mayor o igual que 1")

    operadores = {
        "intercambio": vecino_intercambio_rs,
```

```

        "inversión": vecino_inversion_rs,
        "inserción": vecino_insercion_rs
    }

    mejor_vecina = None
    mejor_distancia_vecina = float("inf")
    mejor_operador = None

    nombres_operadores = list(operadores.keys())

    for candidato in range(cantidad_candidatos):
        # Los tres primeros candidatos garantizan que se pruebe cada operador.
        # Los restantes operadores se seleccionan aleatoriamente.
        if candidato < len(nombres_operadores):
            nombre = nombres_operadores[candidato]
        else:
            nombre = random.choice(nombres_operadores)

        vecina = operadores[nombre](solucion)
        distancia_vecina = distancia_total(vecina, problema)

        if distancia_vecina < mejor_distancia_vecina:
            mejor_vecina = vecina
            mejor_distancia_vecina = distancia_vecina
            mejor_operador = nombre

    return mejor_vecina, mejor_distancia_vecina, mejor_operador

```

```

[58]: def recocido_simulado_vecinos_mejorados(
        problema, temperatura, cantidad_candidatos=6, temperatura_minima=0.
        ↪0001):
    """
    Recocido simulado que mejora la selección del vecino generando varios
    candidatos en cada iteración. No reemplaza la versión original.
    """
    nodos_problema = list(problema.get_nodes())

    # Solución inicial y mejor solución conocida.
    solucion_referencia = crear_solucion(nodos_problema)
    distancia_referencia = distancia_total(solucion_referencia, problema)

    mejor_solucion = solucion_referencia.copy()
    mejor_distancia = distancia_referencia
    distancia_inicial = distancia_referencia

    iteraciones = 0
    soluciones_aceptadas = 0

```

```

peores_aceptadas = 0
uso_operadores = {"intercambio": 0, "inversión": 0, "inserción": 0}

while temperatura > temperatura_minima:
    iteraciones += 1

    # Se generan varios vecinos y se selecciona el mejor candidato.
    vecina, distancia_vecina, operador = seleccionar_vecino_mejorado(
        solucion_referencia, problema, cantidad_candidatos
    )
    uso_operadores[operador] += 1

    # Se actualiza la mejor solución global.
    if distancia_vecina < mejor_distancia:
        mejor_solucion = vecina.copy()
        mejor_distancia = distancia_vecina

    delta = distancia_vecina - distancia_referencia

    # Se aceptan siempre las mejoras. Las soluciones peores pueden
    # aceptarse según la temperatura y la diferencia de distancia.
    if delta < 0 or probabilidad(temperatura, delta):
        if delta > 0:
            peores_aceptadas += 1

            solucion_referencia = vecina
            distancia_referencia = distancia_vecina
            soluciones_aceptadas += 1

    temperatura = bajar_temperatura(temperatura)

estadisticas = {
    "distancia_inicial": distancia_inicial,
    "distancia_final": mejor_distancia,
    "iteraciones": iteraciones,
    "soluciones_aceptadas": soluciones_aceptadas,
    "soluciones_peores_aceptadas": peores_aceptadas,
    "uso_operadores": uso_operadores
}

return mejor_solucion, mejor_distancia, iteraciones, estadisticas

# Ejecución de la versión mejorada
temperatura_inicial = 100000
cantidad_candidatos = 6

```

```

mejor_solucion_rs, mejor_distancia_rs, iteraciones_rs, estadisticas_rs = (
    recocido_simulado_vecinos_mejorados(
        problema,
        temperatura_inicial,
        cantidad_candidatos=cantidad_candidatos
    )
)

print("Recocido simulado con selección mejorada de vecinos")
print(f"Distancia inicial: {estadisticas_rs['distancia_inicial']}")
print(f"Mejor distancia: {mejor_distancia_rs}")
print(f"Iteraciones: {iteraciones_rs}")
print(f"Soluciones aceptadas: {estadisticas_rs['soluciones_aceptadas']}")
print(f"Soluciones peores aceptadas: {estadisticas_rs['soluciones_peores_aceptadas']}")
print(f"Operadores seleccionados: {estadisticas_rs['uso_operadores']}")
print(f"Mejor solución: {mejor_solucion_rs}")

plot_tsp_solution(problema.edge_weights, mejor_solucion_rs)

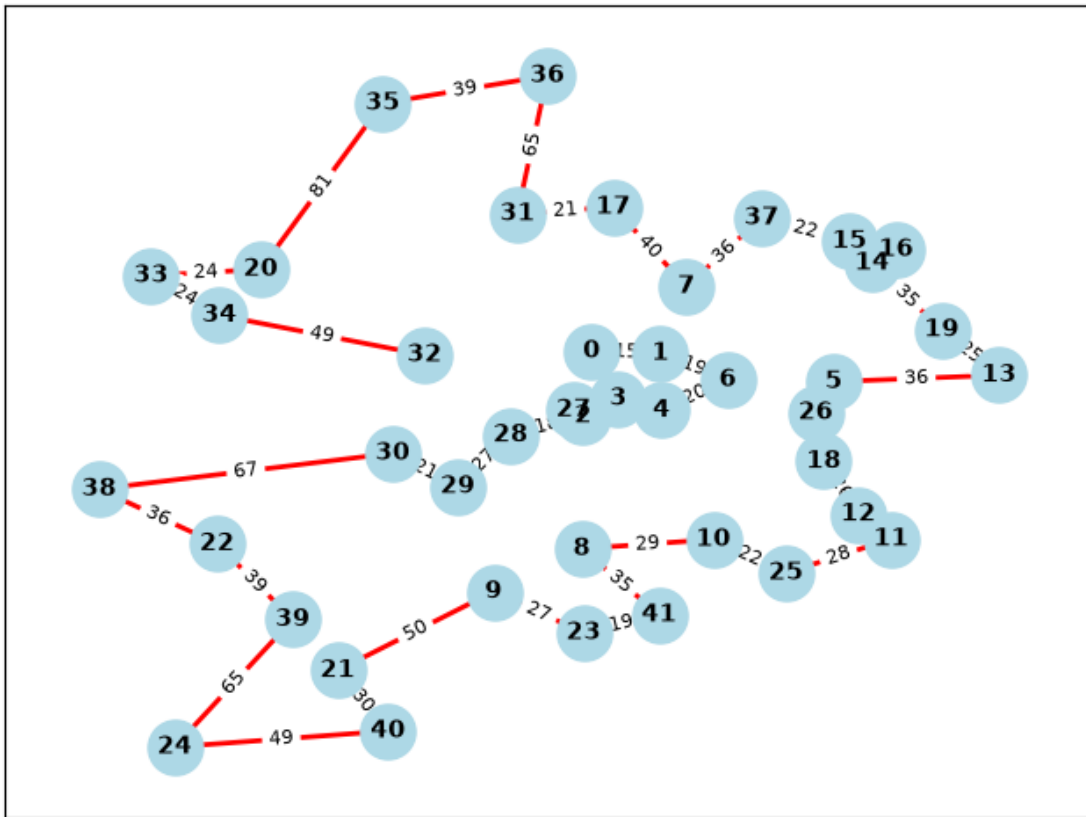
```

```

Recocido simulado con selección mejorada de vecinos
Distancia inicial: 4787
Mejor distancia: 1273
Iteraciones: 207223
Soluciones aceptadas: 79599
Soluciones peores aceptadas: 38259
Operadores seleccionados: {'intercambio': 45826, 'inversión': 87904,
'inserción': 73493}
Mejor solución: [0, 1, 6, 4, 3, 27, 2, 28, 29, 30, 38, 22, 39, 24, 40, 21, 9,
23, 41, 8, 10, 25, 11, 12, 18, 26, 5, 13, 19, 14, 16, 15, 37, 7, 17, 31, 36, 35,
20, 33, 34, 32]

```

Grafo TSP con solución específica



Interpretación

La selección mejorada explora más de una alternativa antes de decidir qué vecino evaluar. Los operadores de intercambio, inversión e inserción producen modificaciones diferentes sobre la ruta, aumentando la diversidad de soluciones candidatas. Aunque se escoge el mejor vecino del conjunto generado, el algoritmo conserva el criterio probabilístico del recocido simulado, por lo que todavía puede aceptar temporalmente soluciones peores para escapar de óptimos locales. El parámetro **cantidad_candidatos** controla el equilibrio entre calidad y costo computacional: un valor mayor permite revisar más vecinos por iteración, pero requiere más cálculos de distancia.

Conclusión

El recocido simulado con selección mejorada de vecinos obtuvo una reducción significativa en la distancia total de la ruta, pasando de una solución inicial con distancia 4787 a una mejor solución con distancia 1273, lo que representa una mejora aproximada del 73,4 %. Durante las 207.223 iteraciones se aceptaron 79.599 soluciones, de las cuales 38.259 correspondieron a soluciones temporalmente peores. Esta aceptación controlada permitió que el algoritmo escapara de posibles óptimos locales y explorara distintas regiones del espacio de búsqueda. En cuanto a los operadores utilizados para generar vecinos, la inversión fue seleccionada con mayor frecuencia, seguida por la inserción y el intercambio. Esto evidencia que la generación de múltiples candidatos mediante distintos operadores de vecindad mejora la exploración y permite obtener soluciones de alta calidad.

3.3 Referencias

- Camacho, José M. (2026a). AG3 – Actividad Guiada 3. 03MIAR – Algoritmos de optimización [Archivo PDF]. Campus Virtual VIU. universidadviu.com
- Camacho, José M. (2026b). VC6 – Algoritmos genéticos. 03MIAR – Algoritmos de optimización [Archivo PDF]. Universidad Internacional de Valencia.
- Camacho, José M. (2026c). VC5 – Algoritmos Heurísticos. 03MIAR – Algoritmos de optimización [Archivo PDF]. Universidad Internacional de Valencia.
- Camacho, José M. (2026d). AG3 – Actividad Guiada 3 (2ª parte). 03MIAR – Algoritmos de optimización [Archivo PDF]. Universidad Internacional de Valencia.

3.3.1 Declaración de uso de IA

Se ha utilizado herramienta de IA generativa para formatear la citación a formato APA, revisar bibliografía, revisar algunas ortografías, formatear formulas en Markdown, y como herramienta consultiva para revisar instrucciones y/o errores del uso de sintaxis para Python.